

DALSILENS: ALAT BANTU MOBILE REAL-TIME UNTUK PENDERITA DIKROMASI MENGUNAKAN DALTONISASI LMS DAN IDENTIFIKASI WARNA BERBASIS HSV

Anaf Naufalian¹, Bonang Waspadadi Ligar²

Informatika^{1,2}

Universitas Gunadarma

anafnaufalian@dalsicore.com¹, bonangligar@staff.gunadarma.ac.id²

Abstrak

Penyandang *Color Vision Deficiency* (CVD) tipe Dikromasi (*Protanopia*, *Deutanopia*, dan *Tritanopia*) mengalami kesulitan membedakan warna tertentu, sementara solusi yang tersedia seperti kacamata kromatik relatif mahal dan sulit dijangkau, sedangkan sebagian aplikasi *smartphone* yang ada masih memproses gambar secara statis (bukan *real-time*) atau membebani CPU akibat kalkulasi matriks warna secara manual. Penelitian ini merancang dan mengimplementasikan aplikasi Android bernama Dalsilens yang memanfaatkan kamera *smartphone* secara *real-time* untuk membantu penyandang dikromasi. Aplikasi menerapkan algoritma *LMS Daltonization* untuk mensimulasikan dan mengoreksi warna yang hilang, serta fitur identifikasi nama warna berbasis ruang warna HSV dengan pendekatan *rule-based thresholding*. Transformasi warna dieksekusi melalui *ColorMatrix* yang diakselerasi GPU agar tidak membebani CPU. Hasil pengujian *Black Box Testing* terhadap 15 skenario fungsional menunjukkan seluruhnya **Berhasil**. Pengujian akurasi klasifikasi nama warna mencapai 90% pada pencahayaan normal dan 50% pada pencahayaan redup. Pengujian performa *real-time* menunjukkan rata-rata 57,2 FPS dengan latensi 17,5 ms per *frame*, penggunaan RAM rata-rata 218,54 MB tanpa indikasi *memory leak*, serta utilisasi CPU relatif hanya 19,21% dari kapasitas total perangkat. Pengujian kebergunaan menggunakan *System Usability Scale* (SUS) terhadap 18 responden menghasilkan skor rata-rata 68,33 (Kategori B/Good). Hasil ini menunjukkan bahwa Dalsilens berpotensi menjadi alat bantu visual yang ringan, mandiri (*on-device*), dan mudah diakses bagi penyandang dikromasi.

Kata Kunci: *Color Vision Deficiency, Dikromasi, LMS Daltonization, Ruang Warna HSV, Android, Real-Time.*

Abstract

Individuals with Color Vision Deficiency (CVD) of the Dichromacy type (Protanopia, Deutanopia, and Tritanopia) have difficulty distinguishing certain colors, while available solutions such as chromatic glasses are relatively expensive and hard to access, and many existing smartphone applications still process images statically (not in real-time) or burden the CPU due to manual color matrix calculations. This research designs and implements an Android application called Dalsilens that utilizes the smartphone camera in real-time to assist individuals with dichromacy. The application applies the LMS Daltonization algorithm to simulate and correct missing colors, along with an HSV-based color name identification feature using a rule-based thresholding approach. Color transformation is executed through a GPU-accelerated ColorMatrix to avoid burdening the CPU. Black Box Testing results on 15 functional scenarios show that all scenarios were **Successful**. Color classification accuracy testing reached 90% under normal lighting and 50% under dim lighting. Real-time performance testing shows an average of 57.2 FPS with a latency of 17.5 ms per frame, an average RAM usage of 218.54 MB with no indication of memory leaks, and a relative CPU utilization of only 19.21% of the device's total capacity. Usability testing using the System Usability Scale (SUS) on 18 respondents produced an average score of 68.33 (Category B/Good). These results indicate that Dalsilens has the potential to become a lightweight, on-device, and accessible visual aid for individuals with dichromacy.

Keywords: *Color Vision Deficiency, Dichromacy, LMS Daltonization, HSV Color Space, Android, Real-Time.*

1. PENDAHULUAN

Penglihatan merupakan indra utama pada manusia yang berperan dalam memproses informasi dari lingkungan sekitar. Beberapa penelitian menyebutkan bahwa sekitar 80% informasi yang diterima dan diproses oleh otak manusia berasal dari sistem visual [1], yang menunjukkan bahwa indra penglihatan berperan sangat penting untuk memahami lingkungan sekitar [2]. Namun, tidak semua orang memiliki kemampuan ini karena adanya *Color Vision Deficiency* (CVD) atau buta warna. Penyandang buta warna, khususnya *Protanopia* dan *Deuteranopia*, sering kali mengalami kesulitan membedakan warna tertentu seperti merah (*Protanopia*) dan hijau (*Deuteranopia*) [3]. Kesulitan ini dapat memengaruhi kemampuan individu dalam membedakan warna, seperti lampu lalu lintas, kode warna pada perangkat elektronik, hingga cairan.

Berbagai solusi untuk permasalahan ini sebenarnya sudah tersedia, salah satunya kacamata kromatik yang dirancang secara khusus untuk menyaring panjang gelombang cahaya tertentu yang tumpang tindih antara warna merah dan hijau [4]. Namun, penggunaan kacamata kromatik masih memiliki beberapa keterbatasan, seperti harganya yang relatif tinggi dan ketersediaannya yang terbatas.

Seiring berkembangnya teknologi *mobile*, *smartphone* kini memiliki kemampuan komputasi yang tinggi untuk melakukan pemrosesan citra digital (*Digital Image Processing*) secara *real-time*. Berdasarkan data *StatCounter*, salah satu platform yang paling banyak digunakan di seluruh dunia adalah Android. Sistem operasi Android mendominasi pasar perangkat *mobile* secara global dengan persentase sebesar 71,68% pada tahun 2025 [7]. Platform Android juga menyediakan pustaka bawaan untuk memanipulasi matriks warna, yaitu *Color Matrix*, yang memungkinkan modifikasi piksel pada citra secara efisien tanpa memengaruhi performa perangkat [8].

Berdasarkan permasalahan tersebut, penelitian ini bertujuan merancang dan mengimplementasikan sebuah aplikasi alat bantu visual berjudul *Dalsilens*, yaitu Alat Bantu *Mobile Real-Time* untuk Penderita Dikromasi Menggunakan Daltonisasi LMS dan Identifikasi Warna Berbasis HSV. Aplikasi ini dirancang untuk membantu penyandang buta warna membedakan warna pada suatu objek menggunakan kamera *smartphone* secara *real-time*. Salah satu pendekatan algoritma yang diterapkan dalam penelitian ini adalah metode *LMS Daltonization*. Metode ini bekerja dengan mengonversi matriks warna RGB ke ruang warna LMS, yang merepresentasikan respons sel kerucut pada mata manusia. Selanjutnya, sistem menghitung informasi warna yang hilang pada penyandang dikromasi dan menggeser informasi tersebut ke spektrum warna lain yang masih dapat

dilihat, sehingga meningkatkan kontras visual tanpa mengubah struktur atau bentuk objek asli. Solusi ini diharapkan dapat menjadi alternatif alat bantu berbasis perangkat lunak yang bersifat *on-device*, praktis, dan lebih mudah dijangkau dibandingkan solusi optik konvensional.

2. TINJAUAN PUSTAKA

Berbagai pendekatan dan solusi teknologi telah dikembangkan untuk membantu penyandang *Color Vision Deficiency* (CVD) meningkatkan kemampuan diskriminasi warnanya. Sebagai contoh, penelitian [5] melakukan perbandingan berbasis *smartphone* terhadap empat algoritma berbeda, yaitu *LMS Daltonization*, *Color Blind Filter Service* (CBFS), *LAB color corrector*, dan *shifting color*. Setiap algoritma diuji menggunakan sekumpulan citra uji untuk setiap jenis Dikromasi. Proses evaluasi dilakukan menggunakan perangkat lunak simulator buta warna, yaitu *Vischeck* dan *Coblis*, untuk membandingkan perbedaan persepsi warna sebelum dan sesudah proses koreksi, serta menganalisis waktu pemrosesan yang dibutuhkan oleh setiap algoritma. Namun, pengujian pada penelitian tersebut masih berbasis pemrosesan citra statis yang diunggah secara manual sehingga aplikasi yang dikembangkan tidak memproses citra secara *real-time*. Peneliti menjelaskan bahwa pengguna harus mengarahkan kamera, mengambil gambar, kemudian menekan tombol "OK", setelah itu aplikasi baru memproses gambar tersebut dan menampilkannya di layar *smartphone*.

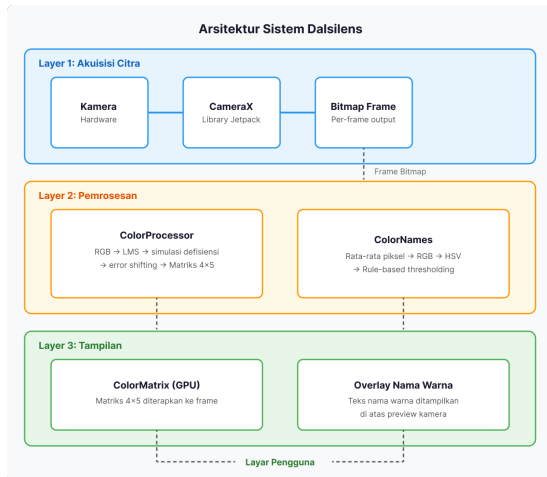
Selain itu, pada penelitian [6], peneliti mengembangkan aplikasi *mobile* yang berfungsi sebagai alat bantu peningkatan penglihatan warna bagi penyandang CVD. Aplikasi tersebut juga memanfaatkan algoritma *LMS Daltonization* untuk mentransformasikan warna citra secara *real-time* menggunakan kamera, serta menerapkan *Farnsworth D-15 Color Arrangement Test* untuk mendiagnosis jenis buta warna yang dialami pengguna secara langsung di dalam aplikasi. Meskipun berhasil menampilkan simulasi secara *real-time*, pemrosesan matriks pada penelitian tersebut masih mengandalkan kalkulasi nilai warna secara manual, sehingga pendekatan komputasi sekuensial ini berpotensi membebani kinerja CPU, terutama pada perangkat dengan resolusi kamera tinggi.

Oleh karena itu, penelitian ini berfokus pada optimalisasi efisiensi komputasi visual *real-time* dengan memindahkan beban operasional dari CPU ke GPU, guna menghindari komputasi sekuensial yang berat. Dengan memanfaatkan pustaka *CameraX* dan manipulasi *ColorMatrix*, *Dalsilens* diharapkan mampu mencapai koreksi warna *real-time* yang stabil, sehingga dapat mengatasi masalah *frame-drop* dan latensi yang sering terjadi pada pemrosesan citra *real-time* konvensional.

3. METODE PENELITIAN

3.1. Arsitektur Sistem

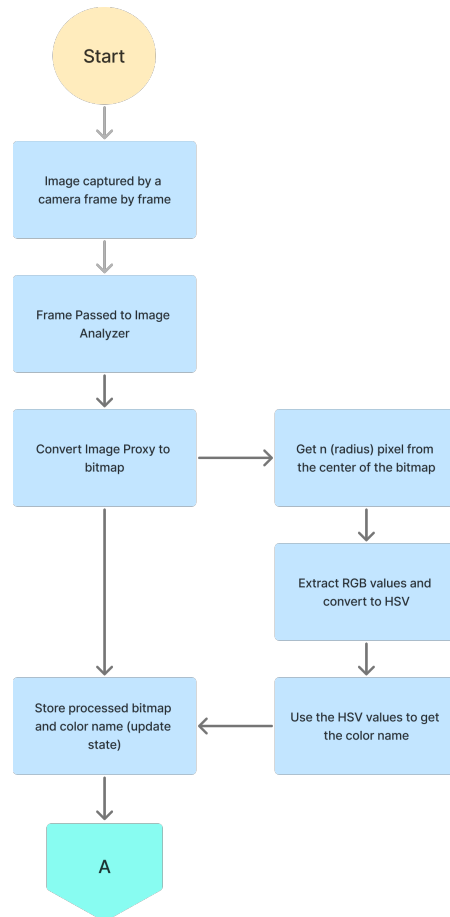
Secara umum, arsitektur sistem Dalsilens dirancang untuk memproses citra secara *real-time* dengan latensi rendah. Komputasi pada sistem ini dibagi menjadi dua tahap utama, yaitu akuisisi citra untuk deteksi warna, dan manipulasi warna pada tingkat *bitmap* menggunakan *color matrix* serta *filtering* yang dipercepat GPU, sebagaimana ditunjukkan pada Gambar 1.



Gambar 1. Arsitektur Umum Sistem Dalsilens

3.1.1 Akuisisi Citra dan Identifikasi Warna

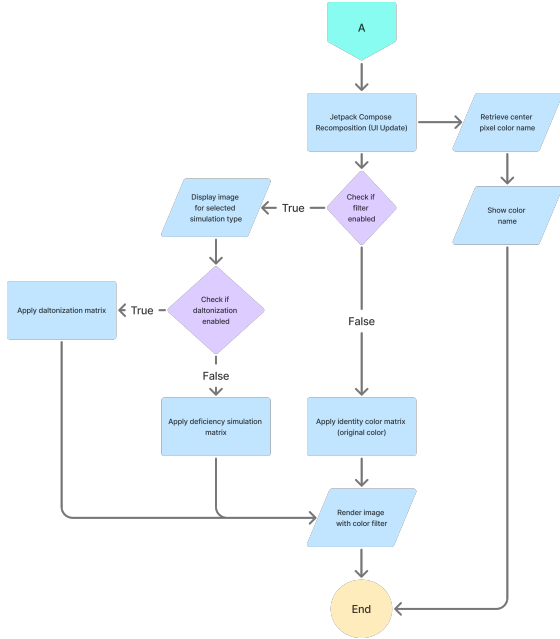
Flowchart pada Gambar 2 menggambarkan proses awal pengolahan citra yang ditangkap oleh perangkat keras kamera. Pertama, ketika pengguna mengakses antarmuka utama aplikasi, sistem secara otomatis menginisialisasi modul kamera. Selanjutnya, kamera secara terus-menerus menangkap setiap *frame* dan meneruskannya ke komponen *ImageAnalysis*. Citra yang diterima komponen ini kemudian dikonversi menjadi representasi *bitmap* mentah untuk diproses lebih lanjut oleh memori sistem. Setelah proses konversi selesai, sistem menentukan area atau *Region of Interest* (ROI) tertentu yang berada tepat di tengah *bitmap*, merepresentasikan objek yang menjadi target *crosshair*. Sistem mengekstrak sekumpulan piksel dalam ROI tersebut untuk dianalisis. Piksel yang diekstrak kemudian dirata-ratakan untuk memperoleh nilai RGB representatif, yang selanjutnya dikonversi ke ruang warna HSV. Nilai HSV yang dihasilkan kemudian diklasifikasikan menggunakan pendekatan *rule-based thresholding* untuk menentukan warna dominan pada area target. Pada tahap akhir, *bitmap* mentah dan label warna dominan yang dihasilkan disimpan dalam variabel *state*. Perubahan pada *state* ini selanjutnya memicu pembaruan antarmuka secara reaktif, sehingga memungkinkan *rendering* visual yang dinamis.



Gambar 2. Flowchart Akuisisi Citra dan Identifikasi Warna

3.1.2 Proses Daltonisasi

Berdasarkan *flowchart* pada Gambar 3, setelah *bitmap* dan label warna diperoleh dan disimpan, sistem memicu mekanisme pembaruan reaktif untuk memperbarui antarmuka pengguna. Perubahan pada label warna yang terdeteksi langsung tercermin pada antarmuka, sementara *bitmap* menjalani tahap verifikasi untuk menentukan apakah filter telah diaktifkan. Jika filter tidak diaktifkan, sistem menerapkan matriks warna identitas untuk mempertahankan representasi warna asli citra. Sebaliknya, jika filter diaktifkan, transformasi matriks warna diterapkan sesuai dengan model simulasi buta warna yang dipilih. Selanjutnya, ketika fitur daltonisasi diaktifkan, citra diproses lebih lanjut menggunakan algoritma daltonisasi berbasis LMS untuk meningkatkan persepsi warna. *Bitmap* dan matriks warna yang telah ditentukan kemudian diteruskan ke komponen *Image*. Pada tahap ini, manipulasi warna dilakukan menggunakan *Color Matrix Filter* yang dieksekusi melalui akselerasi GPU, sehingga proses *rendering* dapat dilakukan secara efisien.



Gambar 3. Flowchart Simulasi Defisiensi Penglihatan Warna dan Proses Daltonisasi

3.2. Implementasi Algoritma LMS Daltonization

Pada bagian ini akan dibahas implementasi algoritma *LMS Daltonization*, dimulai dari konversi RGB ke LMS yang merupakan ruang warna yang digunakan mata manusia untuk mempersepsikan warna, dilanjutkan dengan simulasi kondisi buta warna, hingga proses implementasi algoritma daltonisasi.

3.2.1 Konversi Ruang Warna RGB ke LMS

Tahap pertama dalam proses implementasi algoritma ini adalah mengonversi ruang warna citra dari RGB (*Red, Green, Blue*) ke LMS (*Long, Medium, Short*). Konversi ini dilakukan menggunakan transformasi linear berbasis matriks, yang merepresentasikan tiga jenis sel kerucut pada sistem penglihatan manusia. Secara matematis, transformasi ini dilakukan dengan mengalikan vektor warna RGB dengan matriks konversi tertentu, sehingga menghasilkan representasi warna pada ruang LMS. Nilai L, M, dan S dalam penelitian ini dinyatakan sebagai berikut, dengan koefisien dari [9]:

$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = \begin{bmatrix} 17,8824 & 43,5161 & 4,11935 \\ 3,4565 & 27,1554 & 3,86714 \\ 0,02996 & 0,18431 & 1,46709 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} \quad (1)$$

Koefisien pada persamaan tersebut diperoleh dari [9], yang digunakan sebagai acuan dalam proses konversi warna dari ruang RGB ke LMS. Nilai-nilai ini menunjukkan hubungan antara elemen RGB dan respons setiap kanal pada ruang LMS, sehingga hasil konversi dapat diterapkan secara akurat pada tahap pemrosesan selanjutnya. Ruang warna LMS ini digunakan sebagai dasar

untuk mensimulasikan defisiensi penglihatan warna dan menerapkan algoritma daltonisasi pada tahap berikutnya.

3.2.2 Simulasi Defisiensi Penglihatan Warna

Tahap kedua berfokus pada pemodelan matematis untuk mensimulasikan penurunan kemampuan persepsi warna berdasarkan jenis defisiensi, yaitu *Protanopia*, *Deutanopia*, dan *Tritanopia*. Proses ini dilakukan dengan memodifikasi nilai komponen pada ruang LMS yang diperoleh dari tahap sebelumnya. Pada kondisi ini, salah satu komponen (L, M, atau S) kehilangan sensitivitasnya dan tidak digunakan secara langsung, melainkan dihitung ulang berdasarkan kombinasi dua komponen lainnya, dengan koefisien transformasi dari [10]:

$$\begin{bmatrix} L_{sim} \\ M_{sim} \\ S_{sim} \end{bmatrix}_{Prot} = \begin{bmatrix} 0 & 2,02344 & -2,52581 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} L \\ M \\ S \end{bmatrix} \quad (2)$$

$$\begin{bmatrix} L_{sim} \\ M_{sim} \\ S_{sim} \end{bmatrix}_{Deut} = \begin{bmatrix} 1 & 0 & 0 \\ 0,49421 & 0 & 1,24827 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} L \\ M \\ S \end{bmatrix} \quad (3)$$

$$\begin{bmatrix} L_{sim} \\ M_{sim} \\ S_{sim} \end{bmatrix}_{Trit} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -0,39591 & 0,80111 & 0 \end{bmatrix} \begin{bmatrix} L \\ M \\ S \end{bmatrix} \quad (4)$$

Koefisien pada ketiga persamaan tersebut diperoleh dari [10], [16], yang digunakan untuk menyesuaikan perubahan komponen warna sesuai jenis defisiensi yang disimulasikan. Dengan cara ini, hasil simulasi dapat menunjukkan bagaimana warna tampak bagi pengguna dengan defisiensi penglihatan warna tertentu.

Hasil dari tahap ini berupa nilai LMS yang telah dimodifikasi untuk mensimulasikan kondisi penglihatan pada defisiensi warna. Nilai tersebut kemudian dikonversi kembali ke ruang RGB menggunakan matriks kebalikan (*inverse*) dari Persamaan (1), untuk merepresentasikan bagaimana warna terlihat oleh penyandang buta warna, sekaligus mengamati perubahan hasil setelah penerapan algoritma daltonisasi:

$$\begin{bmatrix} R_{sim} \\ G_{sim} \\ B_{sim} \end{bmatrix} = \begin{bmatrix} 0,08094 & -0,13050 & 0,11672 \\ -0,01025 & 0,05402 & -0,11362 \\ -0,00037 & -0,00412 & 0,69351 \end{bmatrix} \begin{bmatrix} L_{sim} \\ M_{sim} \\ S_{sim} \end{bmatrix} \quad (5)$$

3.2.3 Proses Daltonisasi

Pada tahap ini, proses daltonisasi dilakukan dengan menghitung selisih antara warna asli dan warna

hasil simulasi defisiensi warna. Perhitungan dimulai dengan mengonversi nilai warna dari ruang RGB ke LMS, mensimulasikan defisiensi warna pada ruang LMS, kemudian mengonversinya kembali ke ruang RGB. Selisih warna asli terhadap warna hasil simulasi kemudian dihitung sebagai nilai galat (*error*):

$$E_R = R - R_{sim}, \quad E_G = G - G_{sim}, \quad E_B = B - B_{sim} \quad (6)$$

Setelah nilai galat diperoleh, langkah berikutnya adalah mendistribusikan ulang nilai tersebut ke komponen warna lain melalui proses *error shifting*. Proses ini dilakukan karena komponen warna yang terpengaruh oleh defisiensi tidak dapat dipersepsikan secara optimal oleh pengguna, sehingga informasi warna yang hilang perlu dialihkan ke komponen warna lain yang masih dapat dibedakan. Redistribusi ini dilakukan menggunakan matriks transformasi yang berbeda untuk setiap jenis defisiensi, guna menyesuaikan arah distribusi galat ke komponen warna yang masih dapat dipersepsikan secara optimal.

Pada kondisi *Protanopia*, komponen merah mengalami penurunan sensitivitas sehingga tidak menerima distribusi galat, dan seluruh galat dialihkan ke komponen hijau dan biru. Pada *Deutanopia*, komponen hijau merupakan komponen yang paling terpengaruh sehingga tidak digunakan dalam redistribusi, dan galat dialihkan ke komponen merah dan biru. Sedangkan pada *Tritanopia*, komponen biru mengalami penurunan sensitivitas, sehingga galat dialihkan ke komponen merah dan hijau. Koefisien ketiga matriks berikut diperoleh dari [8], [10], [16]:

$$\begin{bmatrix} R_{shift} \\ G_{shift} \\ B_{shift} \end{bmatrix}_{Prot} = \begin{bmatrix} 0 & 0 & 0 \\ 0,7 & 1 & 0 \\ 0,7 & 0 & 1 \end{bmatrix} \begin{bmatrix} E_R \\ E_G \\ E_B \end{bmatrix} \quad (7)$$

$$\begin{bmatrix} R_{shift} \\ G_{shift} \\ B_{shift} \end{bmatrix}_{Deut} = \begin{bmatrix} 1 & 0,7 & 0 \\ 0 & 0 & 0 \\ 0 & 0,7 & 1 \end{bmatrix} \begin{bmatrix} E_R \\ E_G \\ E_B \end{bmatrix} \quad (8)$$

$$\begin{bmatrix} R_{shift} \\ G_{shift} \\ B_{shift} \end{bmatrix}_{Trit} = \begin{bmatrix} 1 & 0 & 0,7 \\ 0 & 1 & 0,7 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} E_R \\ E_G \\ E_B \end{bmatrix} \quad (9)$$

Nilai galat yang telah digeser kemudian dijumlahkan kembali dengan warna asli untuk menghasilkan warna hasil koreksi (daltonisasi):

$$R_d = R + R_{shift}, \quad G_d = G + G_{shift}, \quad B_d = B + B_{shift} \quad (10)$$

3.2.4 Konstruksi Matriks Transformasi 4×5

Pada tahap ini, seluruh proses transformasi warna yang telah dijelaskan sebelumnya direpresentasikan sebagai fungsi transformasi $T(R, G, B)$. Untuk mengoptimalkan beban komputasi dan menghindari kalkulasi piksel yang berulang, fungsi transformasi ini disederhanakan menjadi sebuah matriks konstan berukuran 4×5. Penyederhanaan dilakukan dengan mengevaluasi fungsi T terhadap tiga vektor warna primer, yaitu merah murni (1,0,0), hijau murni (0,1,0), dan biru murni (0,0,1):

$$\begin{aligned} T(1,0,0) &= (r_R, r_G, r_B) \\ T(0,1,0) &= (g_R, g_G, g_B) \\ T(0,0,1) &= (b_R, b_G, b_B) \end{aligned} \quad (11)$$

Nilai hasil transformasi ketiga vektor basis tersebut kemudian disusun ke dalam kolom-kolom koefisien matriks M_{dalton} , sehingga matriks ini dapat langsung memetakan profil warna asli menjadi warna hasil koreksi, sekaligus mempertahankan parameter *alpha* (transparansi) pada baris keempat:

$$M_{dalton} = \begin{bmatrix} r_R & g_R & b_R & 0 & 0 \\ r_G & g_G & b_G & 0 & 0 \\ r_B & g_B & b_B & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \quad (12)$$

Matriks M_{dalton} inilah yang langsung diterapkan pada *ColorMatrix* Android dan dieksekusi melalui akselerasi GPU, sehingga tidak diperlukan kalkulasi ulang Persamaan (1) hingga (10) pada setiap piksel citra secara *real-time*.

3.3. Implementasi Tingkat Keparahan (Severity Level)

Bagian ini menjelaskan bagaimana penyesuaian tingkat keparahan defisiensi warna diimplementasikan secara dinamis. Parameter *severity* direpresentasikan dalam rentang desimal 0,0 hingga 1,0. Untuk menjaga akurasi representasi visual, sistem membedakan metode penerapan *severity* berdasarkan tujuannya, yaitu antara proses simulasi penglihatan dan proses daltonisasi.

3.3.1 Pemodelan Severity pada Simulasi Defisiensi

Pada mode simulasi, parameter *severity* berfungsi untuk meniru tingkat kerusakan atau pergeseran sensitivitas pada sel kerucut mata pengguna. Proses ini mendekati suatu kondisi medis yang dikenal sebagai *Anomalous Trichromacy* [11]. Oleh karena itu, perhitungan dilakukan secara langsung pada ruang warna LMS menggunakan metode interpolasi linear. Sistem menghitung selisih antara nilai LMS pada kondisi penglihatan normal dan nilai LMS pada kondisi defisiensi penuh, kemudian

selisih tersebut dikalikan dengan parameter *severity* dan dijumlahkan kembali ke nilai normal:

$$L_f = L_n + (L_{sim} - L_n) \times S \quad (13)$$

3.3.2 Pemodelan Severity pada Koreksi Penglihatan Warna

Berbeda dari tahap simulasi, parameter *severity* pada proses daltonisasi berfungsi untuk menentukan intensitas atau kekuatan bantuan warna yang diberikan kepada pengguna. Perhitungan ini dilakukan pada ruang warna RGB, tepatnya pada tahap akhir komputasi ketika nilai galat yang telah digeser ditambahkan ke piksel asli. Intensitas kompensasi warna disesuaikan dengan mengalikan hasil galat yang telah digeser dari setiap kanal warna dengan nilai *severity*, sebelum digabungkan dengan warna asli, sehingga pengguna dapat menyesuaikan kontras warna secara proporsional sesuai kenyamanan visualnya:

$$R_d = R + (R_{shift} \times S) \quad (14)$$

3.4. Ekstraksi Warna Dominan Berbasis Ruang Warna HSV

Pada bagian ini akan dibahas proses ekstraksi warna dominan berdasarkan ruang warna HSV. Tahap ini bertujuan mengekstrak warna dari objek yang berada pada titik tengah yang menjadi target *crosshair* secara *real-time*. Proses ini dimulai dengan memperoleh citra dalam bentuk *bitmap*, dilanjutkan dengan pemrosesan area piksel di sekitar titik tengah, dan diakhiri dengan klasifikasi nama warna menggunakan ruang warna HSV.

3.4.1 Akuisisi dan Pemrosesan Bitmap

Frame dari kamera diperoleh melalui *ImageAnalysis* pada *CameraX*, yang secara terus-menerus menangkap citra. *Frame* tersebut kemudian diteruskan ke fungsi *analyzer*, yang menghasilkan *ImageProxy*, sebelum akhirnya dikonversi menjadi *Bitmap*. Mengingat orientasi sensor kamera bervariasi antar perangkat Android, nilai *rotationDegrees* pada metadata *frame* diperiksa. Apabila nilai rotasi tidak sama dengan nol, *bitmap* akan dirotasi menggunakan matriks transformasi.

3.4.2 Pengambilan Sampel Piksel

Pengambilan sampel warna tidak hanya dilakukan pada satu titik piksel di tengah layar, karena sampel satu piksel cenderung kurang efektif akibat warna pada suatu area dapat dipengaruhi oleh gangguan (*noise*), pencahayaan, atau kondisi lainnya. Oleh karena itu, digunakan pendekatan *sampling* dengan mengambil beberapa piksel di sekitar titik tengah, sehingga warna yang diperoleh menjadi

lebih stabil dan dapat merepresentasikan warna dominan pada area yang diamati dengan lebih baik.

Langkah pertama adalah menentukan radius area *sampling*, dengan nilai radius *default* sebesar $r = 5$. Dengan radius tersebut, area sampel yang diambil mencakup 121 piksel (11×11). Jumlah piksel yang digunakan dapat dihitung menggunakan persamaan berikut:

$$N = (2r + 1) \times (2r + 1) \quad (15)$$

Diberikan citra I berukuran $W \times H$ dan titik tengah (x_c, y_c) dengan radius r , sebuah fungsi pembatas (*bounding function*) didefinisikan agar koordinat piksel yang diproses tidak melampaui batas citra:

$$f(x) = \min(\max(x, 0), W - 1) \quad (16)$$

$$g(y) = \min(\max(y, 0), H - 1) \quad (17)$$

Setiap piksel pada koordinat (x, y) disimpan sebagai bilangan bulat 32-bit berformat ARGB. Struktur bit dari komponen biru (B) berada pada posisi paling akhir, sehingga tidak diperlukan pergeseran bit, cukup dilakukan *masking* untuk menghilangkan bit lainnya menggunakan nilai *mask* 255 (0xFF). Sementara itu, komponen hijau (G) berada pada posisi tengah sehingga perlu digeser ke kanan sebanyak delapan bit terlebih dahulu, dan komponen merah (R) berada pada posisi paling kiri sehingga perlu digeser ke kanan sebanyak 16 bit. Ketiga komponen warna tersebut diekstrak melalui operasi pergeseran bit (*bit shift*) dan *masking* terhadap nilai piksel:

$$B_{(x,y)} = \text{pixel} \& 255 \quad (18)$$

$$G_{(x,y)} = (\text{pixel} \gg 8) \& 255 \quad (19)$$

$$R_{(x,y)} = (\text{pixel} \gg 16) \& 255 \quad (20)$$

Nilai RGB rata-rata dari seluruh N piksel dalam area sampel kemudian dihitung sebagai representasi warna dominan:

$$\bar{R} = \frac{1}{N} \sum_{i=1}^N R_i, \quad \bar{G} = \frac{1}{N} \sum_{i=1}^N G_i, \quad \bar{B} = \frac{1}{N} \sum_{i=1}^N B_i \quad (21)$$

3.4.3 Klasifikasi Nama Warna

Tahap akhir adalah klasifikasi warna, yang dilakukan dengan mengonversi nilai RGB rata-rata yang diperoleh ke ruang warna HSV menggunakan fungsi bawaan Android pada kelas `android.graphics.Color`, yaitu `RGBToHSV(r, g, b, hsv)`. Hasil konversi terdiri dari tiga komponen, yaitu *Hue* (H) sebagai jenis warna pada rentang 0° – 360° , *Saturation* (S) sebagai tingkat intensitas warna pada rentang 0 hingga 1, dan *Value* (V) sebagai tingkat kecerahan pada rentang 0 hingga 1. Ketiga komponen ini kemudian digunakan untuk menentukan nama warna menggunakan pendekatan

threshold berdasarkan rentang nilai H, S, dan V. Nilai *threshold* ditentukan melalui pengujian sampel warna pada kondisi pencahayaan normal dan redup, dengan mempertimbangkan nilai rata-rata, simpangan baku, dan rentang toleransi.

Sebagai contoh, berikut perhitungan untuk menentukan rentang nilai H bagi warna merah. Data uji diperoleh dari 10 sampel pada kondisi pencahayaan normal dan 10 sampel pada kondisi pencahayaan redup, yang kemudian digunakan untuk menghitung nilai rata-rata dan rentang variasi komponen *Hue*. Hasil perhitungan menunjukkan bahwa nilai rata-rata komponen *Hue* pada kondisi pencahayaan normal adalah $3,3^\circ$, sedangkan pada kondisi pencahayaan redup adalah $8,1^\circ$. Selanjutnya, dihitung simpangan baku untuk mengetahui sebaran data pada masing-masing kondisi, dan diperoleh nilai simpangan baku sebesar $0,48^\circ$ pada kondisi normal dan $0,88^\circ$ pada kondisi redup.

Nilai toleransi (batas atas dan bawah) kemudian ditentukan dengan menjumlahkan dan mengurangi nilai rata-rata dengan simpangan baku, menggunakan persamaan $\bar{H} \pm 2\sigma$. Batas bawah untuk kondisi pencahayaan normal diperoleh dari $3,3 - 2(0,48^\circ) = 2,34^\circ$, sedangkan batas atas diperoleh dari $3,3 + 2(0,48^\circ) = 4,26^\circ$, sehingga rentang toleransi pada kondisi normal berada di antara $2,34^\circ$ dan $4,26^\circ$. Metode yang sama diterapkan pada kondisi redup, dengan batas bawah $8,1 - 2(0,88^\circ) = 6,34^\circ$ dan batas atas $8,1 + 2(0,88^\circ) = 9,86^\circ$, sehingga rentang toleransi pada kondisi redup berada di antara $6,34^\circ$ dan $9,86^\circ$.

Apabila dibandingkan, rentang nilai *Hue* dari kondisi normal dan redup tidak menunjukkan tumpang tindih (*overlap*). Berdasarkan hasil ini, penentuan nilai *threshold* dilakukan dengan menetapkan batas atas dari kondisi dengan variasi terbesar, yaitu $9,86^\circ$, sehingga nilai *threshold* untuk warna merah dinyatakan sebagai $H < 9,86^\circ$. Metode yang sama diterapkan pada warna lainnya, sehingga menghasilkan tabel *threshold* yang dapat langsung digunakan pada proses klasifikasi warna, sebagaimana ditunjukkan pada Tabel 1.

Rentang Hue ($^\circ$)	Kategori Warna
$H < 9,85$	Merah
$9,85 \leq H < 26,98$	Oranye
$26,98 \leq H < 50,42$	Kuning
$50,42 \leq H < 122,50$	Hijau
$122,50 \leq H < 204,85$	Cyan
$204,85 \leq H < 224,57$	Biru
$224,57 \leq H < 319,37$	Ungu
$H \geq 319,37$	Merah

Tabel 1. Klasifikasi Warna Berdasarkan Rentang Hue

Warna dengan nilai *Saturation* yang rendah tidak diklasifikasikan menggunakan *Hue* karena cenderung bersifat akromatik (*grayscale*), sehingga klasifikasinya beralih menggunakan nilai *Value* untuk membedakan kategori Hitam, Abu-abu Gelap,

Abu-abu, dan Putih, sebagaimana ditunjukkan pada Tabel 2.

Saturation	Value	Kategori
$S < 0,10$	$V < 0,15$	Hitam
$S < 0,10$	$0,15 \leq V < 0,40$	Abu-abu Gelap
$S < 0,10$	$0,40 \leq V < 0,78$	Abu-abu
$S < 0,10$	$V \geq 0,78$	Putih

Tabel 2. Klasifikasi Warna Akromatik Berdasarkan Saturation dan Value

Dengan menggabungkan klasifikasi berbasis *Hue*, *Saturation*, dan *Value*, sistem mampu mengenali warna kromatik maupun akromatik secara *real-time*, meskipun kondisi pencahayaan lingkungan bervariasi.

3.5. Skenario Pengujian

Pengujian aplikasi mencakup empat skenario, yaitu (1) pengujian fungsionalitas menggunakan *Black Box Testing* [12] terhadap 15 skenario penggunaan; (2) pengujian akurasi klasifikasi nama warna terhadap 10 kategori warna pada kondisi pencahayaan normal dan redup; (3) pengujian performa *real-time* berupa FPS, latensi, penggunaan RAM, dan utilisasi CPU menggunakan *adb shell*, *dumpsys* dan *Android Studio Profiler*; serta (4) pengujian kebergunaan menggunakan kuesioner *System Usability Scale* (SUS) [13] terhadap 18 responden.

4. HASIL DAN PEMBAHASAN

4.1. Pengujian Fungsionalitas

Pengujian *Black Box Testing* [12] dilakukan terhadap 15 skenario yang mencakup navigasi antarmuka, aktivasi/nonaktivasi filter, pemilihan jenis simulasi (*Protanopia*, *Deutanopia*, *Tritanopia*), pengaturan *severity*, pengaturan ukuran area *sampling*, serta deteksi nama warna secara *real-time*. Setiap skenario diuji dengan memberikan masukan tertentu dan memverifikasi kesesuaian keluaran terhadap ekspektasi yang dirancang, mulai dari navigasi antarmuka, aktivasi dan nonaktivasi filter, pemilihan ketiga jenis simulasi buta warna, pengaturan *severity* pada nilai minimum dan maksimum, hingga pengaturan ukuran area *sampling*. Seluruh 15 skenario tersebut menghasilkan keluaran **Berhasil**, yang menunjukkan bahwa fitur-fitur utama aplikasi berjalan sesuai dengan ekspektasi rancangan tanpa ditemukan kegagalan fungsional.

4.2. Pengujian Akurasi Klasifikasi Warna

Pengujian dilakukan terhadap 10 objek fisik yang mewakili setiap kategori warna, pada dua kondisi pencahayaan yaitu **normal** (cahaya ruangan standar) dan **redup** (cahaya dikurangi menggunakan tirai). Hasil deteksi nama warna dibandingkan dengan warna objek yang sebenarnya, sebagaimana ditunjukkan pada Tabel 3 dan Tabel 4.

No.	Objek	Asli	Deteksi	Ket.
1	Tali	Merah	Merah	Benar
2	Baju	Oranye	Oranye	Benar
3	Baju	Kuning	Kuning	Benar
4	Teko	Hijau	Hijau	Benar
5	Pembersih Keyboard	Cyan	Cyan	Benar
6	Gantungan Kunci	Biru	Biru	Benar
7	KTM	Ungu	Magenta	Salah
8	Tumbler	Putih	Putih	Benar
9	Celana	Hitam	Hitam	Benar
10	Gagang Jemuran	Abu-abu	Abu-abu	Benar

Tabel 3. Hasil Pengujian Klasifikasi Warna pada Pencahayaan Normal

No.	Objek	Asli	Deteksi	Ket.
1	Tali	Merah	Merah	Benar
2	Baju	Oranye	Oranye	Benar
3	Baju	Kuning	Oranye	Salah
4	Teko	Hijau	Hijau	Benar
5	Pembersih Keyboard	Cyan	Biru	Salah
6	Gantungan Kunci	Biru	Biru	Benar
7	KTM	Ungu	Merah	Salah
8	Tumbler	Putih	Abu-abu	Salah
9	Celana	Hitam	Hitam	Benar
10	Gagang Jemuran	Abu-abu	Kuning	Salah

Tabel 4. Hasil Pengujian Klasifikasi Warna pada Pencahayaan Redup

Berdasarkan Tabel 3 dan Tabel 4, akurasi deteksi pada kondisi pencahayaan normal sebesar $\frac{9}{10} \times 100\% = 90\%$, dengan satu kesalahan pada warna ungu yang terdeteksi sebagai magenta. Pada kondisi pencahayaan redup, akurasi menurun menjadi $\frac{5}{10} \times 100\% = 50\%$, sebagaimana dirangkum pada Tabel 5.

Kondisi Pencahayaan	Benar	Akurasi
Normal	9/10	90%
Redup	5/10	50%

Tabel 5. Rekapitulasi Akurasi Klasifikasi Nama Warna

Penurunan akurasi pada kondisi redup terjadi karena nilai *Value* (V) pada ruang warna HSV menurun secara signifikan, sehingga piksel yang seharusnya terklasifikasi berwarna cenderung terdeteksi sebagai abu-abu atau hitam. Temuan ini konsisten dengan penelitian [14] yang menyatakan bahwa akurasi kolorimetri berbasis *smartphone* pada ruang warna HSV sensitif terhadap kondisi pencahayaan.

4.3. Pengujian Performa Real-Time

Pengujian performa dilakukan pada skenario beban komputasi tertinggi, yaitu saat mode daltonisasi dan identifikasi warna berjalan bersamaan, merepresentasikan kondisi *worst-case* yang mungkin terjadi. Pengujian FPS dan latensi dilakukan menggunakan data statistik *rendering frame* dari perintah `adb shell dumpsys gfxinfo`, sebanyak tiga kali iterasi, masing-masing berjalan kurang lebih satu menit, sebagaimana ditunjukkan pada Tabel 6.

Iterasi	Frame	FPS	Latensi (ms)	Janky (%)
1	2.389	55,0	18,2	28,59
2	2.808	65,3	15,3	9,62
3	1.134	46,8	21,4	23,10
Rata-rata	6.331	57,2	17,5	19,19

Tabel 6. Hasil Pengujian FPS dan Latensi Pemrosesan

Nilai rata-rata 57,2 FPS dengan latensi 17,5 ms menunjukkan bahwa aplikasi mendekati kondisi *real-time* yang mulus (60 FPS) meskipun transformasi *color matrix* dan klasifikasi warna berjalan bersamaan. Data *GPU Rendering Profile* menunjukkan median waktu pemrosesan GPU berada pada kisaran 5–7 ms, jauh lebih rendah dibandingkan total waktu *render per frame*, mengindikasikan bahwa transformasi warna bukan titik hambat (*bottleneck*) utama.

Penggunaan memori diukur melalui *Total Proportional Set Size* (PSS) menggunakan perintah `adb shell dumpsys meminfo`, sebagaimana ditunjukkan pada Tabel 7.

Iterasi	PSS (KB)	PSS (MB)
1	223.147	217,92
2	227.867	222,53
3	220.339	215,18
Rata-rata	223.784	218,54

Tabel 7. Hasil Pengujian Penggunaan Memori (RAM)

Nilai PSS pada tiga iterasi pengujian berada dalam rentang sempit ($\pm 3,5$ MB dari rata-rata) tanpa menunjukkan tren kenaikan yang konsisten antar iterasi, sehingga tidak ditemukan indikasi *memory leak* pada aplikasi Dalsilens.

Utilisasi CPU diukur menggunakan perintah `adb shell top -n 2 -d 1`, dengan nilai yang dicatat berupa sampel kedua dari setiap pasangan pengukuran, sebagaimana ditunjukkan pada Tabel 8.

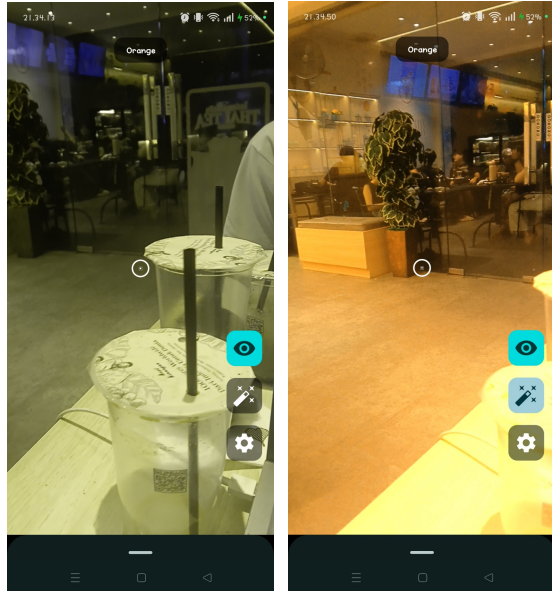
Iterasi	Utilisasi CPU (%)
1	147
2	148
3	166
Rata-rata	153,67

Tabel 8. Hasil Pengujian Utilisasi CPU

Nilai utilisasi CPU melebihi 100% karena utilitas berbasis Linux/Android seperti `top` menghitung persentase CPU sebagai akumulasi penggunaan pada seluruh *core* yang dipakai proses, bukan relatif terhadap satu *core*. Perangkat uji memiliki kapasitas total CPU sebesar 800% (8 *core*), sehingga nilai rata-rata 153,67% apabila dikonversi menjadi persentase relatif terhadap kapasitas total perangkat menghasilkan:

$$\text{Utilisasi CPU Relatif} = \frac{153,67\%}{8 \times 100\%} \times 100\% \approx 19,21\% \quad (22)$$

Nilai ini menunjukkan bahwa aplikasi Dalsilens hanya menggunakan sekitar 19,21% dari total kapasitas komputasi CPU perangkat meskipun menjalankan transformasi *color matrix* dan klasifikasi warna secara bersamaan, membuktikan efektivitas pemindahan beban komputasi berat dari CPU ke GPU melalui *ColorMatrix* [8].



Simulasi Deuteranopia

Simulasi + Daltonisasi

Gambar 4. Perbandingan Tampilan Simulasi Deuteranopia Sebelum dan Sesudah Daltonisasi

Sebagaimana ditunjukkan pada Gambar 4, penerapan daltonisasi menggeser warna objek ke rona yang berbeda dari hasil simulasi murni, sehingga membantu penyandang *Deuteranopia* membedakan objek yang sebelumnya tampak serupa.

4.4. Pengujian Kebergunaan (SUS)

System Usability Scale (SUS) adalah kuesioner standar yang dikembangkan oleh Brooke [13] untuk mengukur tingkat kebergunaan (*usability*) suatu sistem secara cepat dan efisien, terdiri dari 10 pernyataan dengan skala *Likert* 1 hingga 5, di mana pernyataan bernomor ganjil bersifat positif dan pernyataan bernomor genap bersifat negatif [15]. Skor SUS dihitung menggunakan persamaan berikut [13]:

$$\text{Skor SUS} = \left(\sum_{i \in \text{ganjil}} (x_i - 1) + \sum_{j \in \text{genap}} (5 - x_j) \right) \times 2,5 \quad (23)$$

dengan x_i adalah nilai jawaban responden untuk pernyataan ke- i , dan hasil skor berada pada rentang 0 hingga 100. Interpretasi skor SUS mengacu pada klasifikasi pada Tabel 9.

Rentang Skor	Nilai	Kategori
> 80,3	A	Excellent
68 – 80,3	B	Good
57 – 68	C	OK
< 57	D	Poor

Tabel 9. Klasifikasi Skor System Usability Scale

Pengujian dilakukan terhadap 18 responden yang mencoba aplikasi Dalsilens secara langsung pada perangkat Android masing-masing selama kurang lebih lima menit, kemudian mengisi kuesioner SUS. Data demografi responden (dianonimkan sebagai R1–R18) disajikan pada Tabel 10.

Resp.	Usia	JK	Resp.	Usia	JK
R1	20	L	R10	20	P
R2	31	L	R11	20	L
R3	21	L	R12	19	L
R4	20	P	R13	19	P
R5	21	P	R14	21	L
R6	21	L	R15	15	L
R7	28	L	R16	20	P
R8	22	L	R17	21	L
R9	20	P	R18	20	L

Tabel 10. Data Demografi Responden Pengujian SUS (JK: Jenis Kelamin, L: Laki-laki, P: Perempuan)

Rekapitulasi jawaban seluruh responden terhadap 10 pernyataan SUS (P1–P10) menghasilkan skor SUS yang bervariasi antara 47,5 hingga 90,0, dengan rata-rata skor sebesar 68,33. Nilai rata-rata tersebut termasuk dalam kategori B (*Good*) menurut Tabel 9. Hasil ini menunjukkan bahwa aplikasi Dalsilens dinilai baik dan dapat diterima oleh pengguna dari sisi kemudahan dan kenyamanan penggunaan, meskipun masih terdapat ruang perbaikan pada aspek antarmuka untuk mencapai kategori *Excellent*.

Secara keseluruhan, hasil pengujian menunjukkan bahwa pendekatan berbasis *ColorMatrix* yang diakselerasi GPU berhasil menjawab tantangan performa yang dihadapi oleh pendekatan kalkulasi manual pada penelitian terdahulu [6], sementara integrasi langsung ke kamera *real-time* mengatasi keterbatasan pemrosesan statis pada [5]. Tingkat *precision* yang tinggi pada sebagian besar kategori warna (Tabel 3) menandakan tingkat kesalahan *false positive* yang minim dalam konteks identifikasi warna, sementara akurasi klasifikasi warna yang menurun pada kondisi redup (Tabel 4) menjadi keterbatasan utama yang perlu diperhatikan pada pengembangan selanjutnya, misalnya melalui pendekatan berbasis *machine learning* yang lebih adaptif terhadap variasi pencahayaan. Kesenjangan antara penelitian akademis dan penerapan praktis juga turut dijemput melalui antarmuka yang intuitif, sehingga pengguna dapat memperoleh umpan balik visual maupun tekstual secara seketika tanpa memerlukan pemrosesan tambahan di luar perangkat.

5. KESIMPULAN

Aplikasi Dalsilens berhasil dikembangkan sebagai alat bantu visual *real-time* berbasis Android bagi penyandang dikromasi, dengan mengintegrasikan algoritma *LMS Daltonization* dan fitur identifikasi nama warna berbasis ruang warna HSV. Seluruh 15 skenario *Black Box Testing* berjalan **Berhasil**, akurasi klasifikasi warna mencapai 90% pada pencahayaan normal namun menurun pada pencahayaan redup, performa *real-time* rata-rata mencapai 57,2 FPS dengan latensi 17,5 ms tanpa indikasi *memory leak*, serta utilisasi CPU relatif hanya 19,21% berkat pemindahan beban komputasi ke GPU melalui *ColorMatrix*. Skor SUS sebesar 68,33 (Kategori B/Good) menunjukkan tingkat kebergunaan yang baik. Penelitian selanjutnya disarankan untuk menerapkan pendekatan berbasis *machine learning* guna meningkatkan akurasi klasifikasi warna pada kondisi pencahayaan yang bervariasi, serta memperluas dukungan ke tipe *Anomalous Trichromacy*.

DAFTAR PUSTAKA

- [1] D. Man and R. Olchawa, 'The Possibilities of Using BCI Technology in Biomedical Engineering', *Advances in Intelligent Systems and Computing*, pp. 30–37, 2018.
- [2] S. Chung and J.-W. Son, 'Visual Perception in Autism Spectrum Disorder: A Review of Neuroimaging Studies', *Soa Chongsoryon Chongsin Uihak*, pp. 105–120, 2020.
- [3] H.-Y. Lin, L.-Q. Chen, and M.-L. Wang, 'Improving Discrimination in Color Vision Deficiency by Image Re-Coloring', *Sensors*, vol. 19, no. 10, 2019.
- [4] E. Patterson *et al.*, 'Effects of color-enhancing glasses on color vision in congenital red-green color deficiencies', *Optics Express*, pp. 31182–31194, 2022.
- [5] L. A. Elrefaei, 'Smartphone Based Image Color Correction for Color Blindness', *International Journal of Interactive Mobile Technologies (iJIM)*, vol. 12, no. 3, pp. 104–119, Jul. 2018, doi: 10.3991/ijim.v12i3.8160.
- [6] G. Tecson, F. Calanda, G. Cayabyab, and F. Reyes Jr, 'Covisance: A Real Time Mobile Recolorization Tool for Aiding Color Vision Deficient Users Utilizing D-15 Color Arrangement Test', in *Proceedings of the 2017 ACM International Conference*, 2017, pp. 95–99.
- [7] StatCounter, 'Mobile Operating System Market Share Worldwide', 2025. [Online]. Available: <https://gs.statcounter.com/os-market-share/mobile/worldwide/2025>.
- [8] Android Developers, 'Hardware Acceleration'. [Online]. Available: <https://developer.android.com/develop/ui/views/graphics/hardware-accel>.
- [9] F. Viénot, H. Brettel, and J. D. Mollon, 'Digital video colourmaps for checking the legibility of displays by dichromats', *Color Research and Application*, vol. 24, pp. 243–252, 1999.
- [10] A. B. Muhammad, J. Z. Maitama, A. Bature, and Z. Yunusa, 'Analysis of LMS Daltonization Algorithm for Adjusting Image Color for Deuteranopia', *Zaria Journal of Electrical Engineering Technology*, vol. 9, no. 1, pp. 128–135, Mar. 2020.
- [11] G. M. Machado, M. M. Oliveira, and L. A. F. Fernandes, 'A Physiologically-based Model for Simulation of Color Vision Deficiency', *IEEE Transactions on Visualization and Computer Graphics*, vol. 15, no. 6, pp. 1291–1298, 2009.
- [12] Dicoding Indonesia, 'Black Box Testing: Pengertian, Manfaat, dan Cara Kerjanya', 2024. [Online]. Available: <https://www.dicoding.com/blog/black-box-testing/>.
- [13] J. Brooke, 'SUS: A quick and dirty usability scale', *Usability Eval. Ind.*, vol. 189, 1995.
- [14] Y. Sirisathitkul, N. Dinmeung, W. Noonsuk, and C. Sirisathitkul, 'Accuracy and precision of smartphone colorimetry: a comparative analysis in RGB, HSV, and CIELAB color spaces for archaeological research', *STAR: Science & Technology of Archaeological Research*, vol. 11, no. 1, 2025, doi: 10.1080/20548923.2024.2444168.
- [15] J. R. Lewis and J. Sauro, 'Can I Leave This One Out? The Effect of Dropping an Item From the SUS', *Journal of Usability Studies*, vol. 13, pp. 38–46, 2017.
- [16] A. Tasnim and M. S. Hasan, 'An improved dynamic daltonization for color-blinds', in *2017 IEEE Region 10 Humanitarian Technology Conference (R10-HTC)*, 2017, pp. 798–801, doi: 10.1109/R10-HTC.2017.8289076.